```
jakka_bhanu@LAPTOP-DCEVSCQ0:~$ mkdir ~/skill2
jakka_bhanu@LAPTOP-DCEVSCQ0:~$ cd ~/skill2
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill2$ nano input_capture.c
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill2$ gcc input_capture.c -o input_capture
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill2$ ./input_capture
Custom input-capture shell. Type a command, press Enter to submit.
Type 'exit' and press Enter to quit.

myshell> helxyz
You typed: "helxyz" (6 characters)
myshell> hello world
You typed: "hello world" (11 characters)
myshell> exit
Exiting shell...
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill2$ cat input_capture.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_INPUT 256

// Switch the terminal into "raw-ish" mode so we get every keystroke
// immediately, instead of waiting for the user to press Enter.
// We keep it simple: turn off canonical mode (line buffering) and echo,
// so WE control what gets displayed and when input is "done".
struct termios original_settings;

void enable_raw_mode() {
    struct termios raw;
    tcgetattr(STDIN_FILENO, &original_settings);  // save original settings
    raw = original_settings;
```

```
void enable_raw_mode() {
    struct termios raw;
    tcgetattr(STDIN_FILENO, &original_settings);  // save original settings
    raw = original_settings;
    raw.c_lflag &= ~(ICANON | ECHO);  // turn off line-buffering and auto-echo
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void disable_raw_mode() {
    // Restore the terminal to how it was before we messed with it
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_settings);
}

int main() {
    char buffer[MAX_INPUT];
    int pos;
    char ch;

    printf("Custom input-capture shell. Type a command, press Enter to submit.\n");
    printf("Type 'exit' and press Enter to quit.\n\n");

    enable_raw_mode();

    while (1) {
        pos = 0;
        memset(buffer, 0, sizeof(buffer));

        printf("myshell> ");
        fflush(stdout);

        // ---- Read keystrokes one at a time until Enter is pressed ----
        while (1) {
```

jakka_bhanu@LAPTOP-DCEVS

```
        // ---- Read keystrokes one at a time until Enter is pressed ----
        while (1) {
            ssize_t n = read(STDIN_FILENO, &ch, 1);
            if (n <= 0) continue;

            if (ch == '\n' || ch == '\r') {
                // ---- Process Enter key ----
                printf("\n");
                break;
            }
            else if (ch == 127 || ch == 8) {
                // ---- Handle Backspace (127 = DEL, 8 = BS) ----
                if (pos > 0) {
                    pos--;
                    buffer[pos] = '\0';
                    // Move cursor back, print space to erase char, move back again
                    printf("\b \b");
                    fflush(stdout);
                }
                // If pos == 0, there's nothing to delete - do nothing (ignore it)
            }
            else if (pos < MAX_INPUT - 1) {
                // ---- Manage input buffer: add this character ----
                buffer[pos++] = ch;
                buffer[pos] = '\0';
                putchar(ch);  // echo it ourselves since terminal ECHO is off
                fflush(stdout);
            }
        }

        // ---- Handle exit condition ----
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting shell...\n");
```

jakka_bhanu@LAPTOP-DCEVS

```
                buffer[pos++] = ch;
                buffer[pos] = '\0';
                putchar(ch);  // echo it ourselves since terminal ECHO is off
                fflush(stdout);
            }
        }

        // ---- Handle exit condition ----
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (pos == 0) {
            continue;  // empty input, just show prompt again
        }

        // ---- Test user interaction: confirm what was captured ----
        printf("You typed: \"%s\" (%d characters)\n", buffer, pos);
    }

    disable_raw_mode();
    return 0;
}
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill2$ ./input_capture
Custom input-capture shell. Type a command, press Enter to submit.
Type 'exit' and press Enter to quit.

myshell> abc123
You typed: "abc123" (6 characters)
myshell> exit
Exiting shell...
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill2$
```